CRITICALCARE-TWIN: An Offline-First Cloud-Edge Patient Digital Twin Prototype for ICU Risk Monitoring

Research and educational prototype – not clinically validated

Dr Roopa M S
Department of CSE
Nitte Meenakshi Institute of Technology (Deemed to be University), Bengaluru - 560064, India
roopa.ms@nmit.ac.in

Khushi V
1NT23CS109
Department of CSE
Nitte Meenakshi Institute of Technology (Deemed to be University), Bengaluru - 560064, India
1nt23cs109.khushi@nmit.ac.in

Dheeraj N
1NT23CS140
Department of CSE
Nitte Meenakshi Institute of Technology (Deemed to be University), Bengaluru - 560064, India
1nt23cs140.dheeraj@nmit.ac.in

P Poornashree Rai
1NT23CS156
Department of CSE
Nitte Meenakshi Institute of Technology (Deemed to be University), Bengaluru - 560064, India
1nt23cs156.poornashree@nmit.ac.in

Khushi R Patel
1NT23CS108
Department of CSE
Nitte Meenakshi Institute of Technology (Deemed to be University), Bengaluru - 560064, India
1nt23cs108.khushi@nmit.ac.in

***Abstract*—Intensive care units generate continuous physiological data, but monitoring tools rarely specify what happens at the boundary between bedside computation and cloud services. This paper presents CRITICALCARE-TWIN, a research prototype that keeps a patient-specific digital twin at the edge and synchronizes only a reduced, pseudonymous state summary to a cloud service. The edge validates observations, updates a persistent physiological state, runs risk, anomaly and short-horizon forecast analytics locally, and writes an auditable snapshot before any network call. A durable SQLite outbox retries undelivered messages in chronological order and the cloud service ingests them idempotently, so a network outage never stops the local twin. Risk estimation uses logistic regression and a class-balanced random forest trained on 91,713 ICU encounters (8.63% hospital deaths) with APACHE-derived fields excluded; the random forest reached ROC-AUC 0.849 and PR-AUC 0.420 on a random 80/20 split, and a MEWS-style rule-based score serves as a baseline. The system is verified by unit, cloud and dashboard tests. It is a research prototype without clinical validation and must not be used to diagnose, triage or manage patients.**

***Index Terms*—Digital twin, cloud-edge computing, intensive care, patient monitoring, early warning score, offline-first synchronization, pseudonymization, mortality prediction.**

I. INTRODUCTION

Intensive care patients are monitored through bedside devices, laboratory systems and electronic health records that often operate as separate platforms. A digital twin, a virtual representation kept synchronized with a physical entity, has been proposed as a way to bring such data together for monitoring, prediction and what-if analysis [1], [2]. Edge computing places computation close to the data source, which reduces latency and dependence on wide-area links [3], [4]. For patient monitoring this matters because local operation should continue when the network does not.

Early warning scores such as the Modified Early Warning Score (MEWS) convert vital signs into a single value using threshold rules [5], and machine learning has been studied as a way to improve the prediction of clinical deterioration [6]. Public critical-care data such as MIMIC-III [7] support this work. In high-stakes clinical settings, interpretability and an honest statement of validation status are essential [8].

Many digital twin prototypes are described mainly at the dashboard level. Fewer specify what happens at the cloud boundary: which data leave the edge, what happens during an outage, and how retransmission avoids duplicate records. This paper addresses those questions in a working prototype and reports its evidence and limits openly. The contributions are:

1) An offline-first cloud-edge twin architecture with a persistent state estimator, local analytics, a durable outbox, ordered retry and idempotent cloud ingestion.

2) A privacy-minimizing synchronization contract: a pseudonymous patient key, a reduced envelope that excludes raw observations and direct identifiers, and isolation of what-if scenario clones from the cloud record.

3) A risk analytics evaluation on a real ICU encounter table of 91,713 records, comparing logistic regression and a class-balanced random forest with a rule-based MEWS-style baseline using ROC-AUC, PR-AUC and Brier score.

4) A verification suite and an explicit statement of scope, limitations and production-hardening requirements.

The rest of the paper is organized as follows. Section II reviews related work, Section III describes the architecture, Section IV the state model and analytics, Section V the implementation, Section VI the evaluation, and Section VII the limitations.

II. RELATED WORK

A. Healthcare digital twins

Surveys of digital twin technology describe synchronization between a physical entity and its virtual model, data integration, and open challenges including

security, privacy and standardization [1], [2]. Our work contributes a concrete treatment of the synchronization boundary for a patient twin, including failure behavior.

B. Edge and cloud computing

Edge computing is motivated by latency, bandwidth and privacy [3], [4]. We apply this by keeping state estimation, analytics and the dashboard at the edge, and using the cloud only for longitudinal retention and multi-edge aggregation.

C. Early warning scores and machine learning

MEWS was validated on medical admissions to a hospital ward [5]. The National Early Warning Score 2 (NEWS2) [9] extends aggregate scoring with oxygen saturation and a consciousness scale. Machine learning models can outperform simple scores for deterioration prediction [6], and MIMIC-III [7] is a widely used source of critical-care data. We use a MEWS-style score as a transparent baseline rather than as a clinical tool.

D. Explainability and governance

Rudin argues for interpretable models in high-stakes decisions [8]. Our dashboard shows feature contributions, which we treat strictly as associations and not as causal effects or clinical explanations.

III. SYSTEM ARCHITECTURE

A. Edge and cloud domains

Fig. 1 shows the architecture. The edge domain owns raw observations, direct patient identifiers, state estimation and low-latency behavior. The cloud domain receives only a pseudonymous patient key, twin ID, version, state vector, derived values, risk output, uncertainty, provenance class and timestamps.

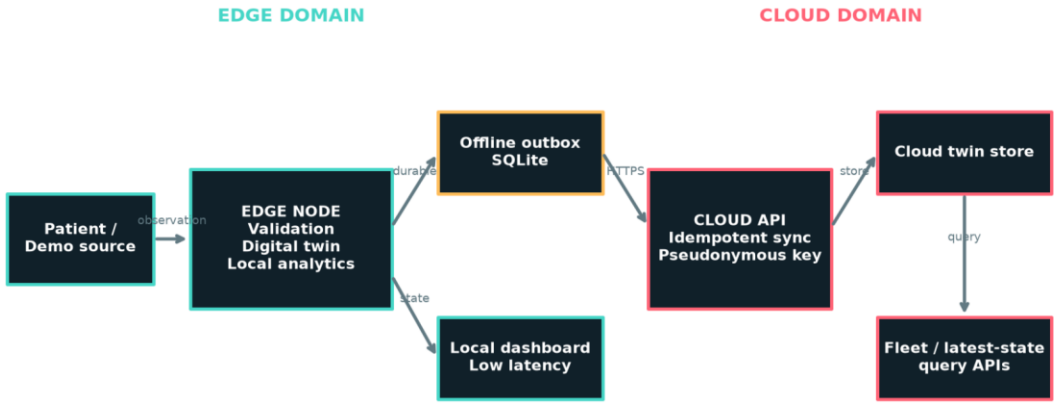


Fig. 1. Cloud-edge architecture and trust boundary. The local demonstration uses HTTP on loopback; a real deployment must terminate TLS and authenticate edge sites.

Edge responsibilities. The edge node ingests and validates observations against configured physiological bounds; maintains the previous state X_{t-1}, the current state X_t, the change ΔX_t, a rolling state history, a patient baseline B and an uncertainty estimate; runs model inference, anomaly scoring and short-horizon forecasting locally; persists patient, observation, event, risk and snapshot records in SQLite; and queues cloud envelopes when the service is unavailable.

Cloud responsibilities. The cloud service accepts idempotent messages keyed by a message identifier while preserving twin version; stores pseudonymous longitudinal state summaries across sessions and edge sites; exposes the latest state for one pseudonymous key and aggregate fleet counts; and remains independent of the presentation layer.

B. Twin lifecycle

At time t the source produces an observation Y_t. The edge validates Y_t and estimates the state

$$X_t = F(X_{t-1}, Y_t, q_t) \quad (1)$$

where q_t is the measurement quality. Derived features

$$Z_t = G(X_t, B, H_t) \quad (2)$$

are computed from the state, the baseline B and the recent state history H_t. They feed the supervised hospital-death model, an independent anomaly detector and a short-horizon forecast. A snapshot is written locally before any cloud envelope is attempted. If the cloud API accepts the envelope, an audit row records success; otherwise the unchanged JSON envelope is placed in the durable outbox.

C. Synchronization contract

The edge derives the patient key as the first 20 hexadecimal characters of SHA-256(salt:patient_id). This is pseudonymization, not anonymization. Each envelope carries a unique message identifier; the cloud store uses INSERT OR IGNORE on that identifier, so retransmission is idempotent. Table I lists the cloud endpoints. Scenario clones created in the What-If Lab explicitly remove their edge gateway, so experimental branches cannot enter the cloud record.

TABLE I
CLOUD API CONTRACT

Endpoint	Purpose
GET /health	Service mode and stored snapshot count

Endpoint	Purpose
POST /v1/twins/sync	Idempotently accept one twin envelope
GET /v1/twins/{patient_key}/latest	Latest version for a pseudonymous patient
GET /v1/fleet/summary	Count of snapshots, pseudonymous patients and sites

D. Persistence and failure behavior

The local twin database stores patients, twins, observations, snapshots, events, risk predictions, simulations and model metadata. A separate edge outbox database stores unsent immutable envelopes and a successful-synchronization audit. The cloud demonstration store indexes messages by patient key and observed time. During an outage, local synchronization continues, outbound messages queue, and the flush retries them in chronological order once the service returns. Duplicate deliveries are absorbed by the idempotent insert.

IV. TWIN STATE MODEL AND RISK ANALYTICS

A. State and uncertainty

Missing values are preserved during preprocessing and are median-imputed only inside the model pipeline. The twin uncertainty separately reflects missing observations and measurement quality. The source table is cross-sectional at encounter level: first-hour and first-day minima and maxima are summaries, not an ordered bedside time series. The time-indexed streams used in the demonstration are therefore simulated replays around an observed baseline, and are never used to fabricate training labels.

B. Supervised hospital-death model

The downstream task predicts the source field hospital_death. Logistic regression and a class-balanced random forest [10] are trained on a fixed 80/20 stratified split of encounters. APACHE mortality probabilities, patient and encounter identifiers, and target-adjacent fields are excluded. Because only 8.63% of encounters are positive, we report PR-AUC alongside ROC-AUC, together with the Brier score. The model output is a hospital-death probability for the dataset task, not a clinical diagnosis.

C. MEWS-style baseline

The rule-based baseline assigns each monitored parameter a score S_i from predefined bands and sums them:

$$\text{MEWS} = \Sigma\, S_i \tag{3}$$

Patients are grouped as Low (score ≤ 2), Moderate (3–4), High (5–6) or Critical (> 6), and alerts are raised for High and Critical. The original MEWS scores systolic blood pressure, heart rate, respiratory rate, temperature and an AVPU consciousness level, and was validated on ward admissions rather than ICU patients [5]. Our variant also scores SpO_2, as NEWS2 does [9], and has no consciousness input, so we call it MEWS-style. The four risk groups are our own grouping and are not taken from [5]. **The implemented scoring bands are: heart rate 51-100=0, 41-50 or 101-110=1, <=40 or 111-129=2, and >=130=3; systolic blood pressure 101-199=0, 81-100=1, 71-80 or >=200=2, and <=70=3; respiratory rate 9-14=0, 15-20=1, <=8 or 21-29=2, and >=30=3; temperature 35.1-38.4=0 and <=35.0 or >=38.5=2; and SpO2 >=96=0, 94-95=1, 92-93=2, and <=91=3.**

D. Anomaly score and forecast

The anomaly score, instability metric, short-horizon forecast and uncertainty layer are prototype methods. Their accuracy has not been evaluated, so we report no performance for them.

V. IMPLEMENTATION

The system is written in Python. SQLite [11] provides persistence for the local twin, the edge outbox and the cloud demonstration store. The dashboard uses Streamlit [12] and has seven pages: Command Center, Live Twin, State Explorer, Risk & Explainability, What-If Lab, Cloud & Edge, and Evidence Lab. The Cloud & Edge page shows live service health, queue depth, synchronization count and the deployment boundary; Evidence Lab combines data provenance and model evaluation. The cloud service is implemented with FastAPI [13].

For development, the cloud boundary and the edge/dashboard run as two processes so that both failure modes are observable: python scripts/run_cloud.py starts the cloud service (port 8000), and python -m streamlit run app.py starts the edge and dashboard (port 8501). The edge target defaults to http://127.0.0.1:8000. The same envelope and retry behavior apply if this URL is replaced by a TLS endpoint, but the evaluated configuration is local. **No hosted cloud deployment was evaluated; the reported latency was measured on the local loopback configuration.**

VI. EXPERIMENTAL EVALUATION

A. Dataset

The primary source is an ICU encounter table with 91,713 rows and 85 columns [14]. The documented hospital-death rate is 8.63%. One column is entirely empty and carries no information; among the remaining columns, the highest missingness is about 10% (day-one potassium and first-hour non-invasive mean blood pressure). Other downloaded datasets are kept separate because no scientifically defensible common patient identifier exists. Distributions and correlations are descriptive and may reflect treatment, selection, site and coding effects.

B. Model results

Table II and Fig. 2 summarize the test-set evidence. The selected random forest reached ROC-AUC 0.849 and PR-AUC 0.420, roughly five times the positive rate of 0.0863. In Fig. 2 the ROC curves of the two models are close, with the random forest slightly higher. These are internal metrics from a random split and do not establish performance at a new site or time.

TABLE II
TEST-SET RESULTS (20% STRATIFIED SPLIT)

Model	ROC-AUC	PR-AUC	Brier
Logistic regression	**0.839**	**0.399**	**0.163**

Model	ROC-AUC	PR-AUC	Brier
Random forest (balanced)	0.849	0.420	**0.132**
MEWS-style baseline	**0.741**	**0.254**	n/a

The MEWS-style baseline was evaluated on the same test split as the statistical models.

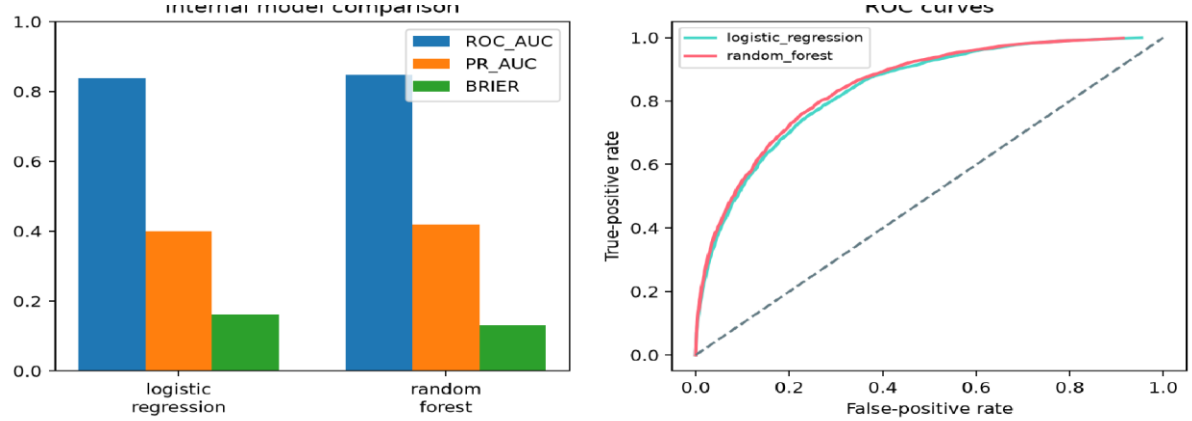


Fig. 2. Internal model comparison (ROC-AUC, PR-AUC, Brier) and ROC curves for logistic regression and the class-balanced random forest.

C. MEWS-style baseline comparison

To anchor the rule-based score, we compute it from the admission vital signs in the encounter table (heart rate, blood pressure, respiratory rate, temperature and minimum SpO_2) and evaluate its ROC-AUC and PR-AUC on the same 20% test split as the models. **The MEWS-style baseline achieved ROC-AUC 0.741 and PR-AUC 0.254, below the class-balanced random forest values of 0.849 and 0.420, respectively.**

D. System verification

Unit tests cover initialization, validation, state transitions, history, event generation, features, persistence and scenario isolation. Cloud tests verify sync acceptance, latest-state retrieval, fleet aggregation, offline buffering and the absence of raw patient identifiers in outbound JSON. A dashboard test renders all seven pages and runs a what-if scenario without mutating the actual twin. The full stack was smoke-tested with the cloud service on port 8000 and the dashboard on port 8501. These tests establish functional correctness of the design, not clinical performance.

System properties such as latency and recovery time are not claimed without measurement. Table III defines the measurements to report: each is taken on loopback, with the cloud service stopped for a fixed outage while the edge keeps producing snapshots, then restarted. **Measurements were performed on Windows 11 using an 11th Generation Intel Core i5-1135G7 processor with four physical cores, eight logical processors and 15.8 GB RAM. Synchronization latency was measured across 100 messages. The cloud service was stopped for two seconds while 20 messages were retained in the edge outbox, then flushed in five batches of four after restart. Idempotency was tested with ten resends of one message.**

TABLE III
SYSTEM MEASUREMENTS

Measurement	p50	p95
End-to-end sync latency (ms)	**26.53**	**33.06**
Outbox flush time after outage (s)	**2.98**	**3.74**
Rows stored after duplicate resends	**1**	n/a

VII. LIMITATIONS AND ETHICAL CONSIDERATIONS

This system is a research and educational prototype. It is not a medical device and has no clinical validation, and the cloud service does not change that status. No output should diagnose, prescribe, triage or manage a patient.

The main limitations are: (i) a random split does not establish generalization to a new site or time; (ii) feature importance and displayed contributions are associations, not causal effects; (iii) the source data are encounter-level summaries and the replay telemetry is synthetic, so no bedside time-series performance is claimed; (iv) the anomaly, instability and forecast methods are unevaluated; and (v) the MEWS-style score is a modified variant with our own risk groups.

The privacy design is pseudonymization with a code-default salt and HTTP on loopback, which is suitable only for demonstration. Production use would require mutual TLS or signed credentials per edge site, secret-managed keyed hashing or tokenization, a managed encrypted database with backups and row-level access control, schema versioning, dead-letter queues, observability, rate limits, disaster recovery, and clinical governance with external validation, a safety case and regulatory assessment, consistent with guidance on the ethics and governance of health AI [15].

VIII. CONCLUSION AND FUTURE WORK

We presented CRITICALCARE-TWIN, an offline-first cloud-edge patient digital twin prototype. Its contribution is a specified synchronization boundary: local state and analytics that never depend on the network, a durable outbox with ordered retry, idempotent cloud ingestion, and a reduced pseudonymous envelope. On a real ICU encounter table, a random forest reached ROC-AUC 0.849 and PR-AUC 0.420

in an internal evaluation, and the system is covered by unit, cloud and dashboard tests.

Future work includes external and temporal validation, evaluation of the anomaly and forecast layers on real telemetry, measured latency and recovery on a hosted cloud deployment, interoperability with HL7 FHIR [16], privacy-preserving training across sites using federated learning [17], and prospective clinical evaluation under proper governance.

REFERENCES

[1] A. Fuller, Z. Fan, C. Day, and C. Barlow, "Digital twin: Enabling technologies, challenges and open research," *IEEE Access*, vol. 8, pp. 108952–108971, 2020.

[2] F. Tao, H. Zhang, A. Liu, and A. Y. C. Nee, "Digital twin in industry: State-of-the-art," *IEEE Trans. Ind. Informat.*, vol. 15, no. 4, pp. 2405–2415, 2019.

[3] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, "Edge computing: Vision and challenges," *IEEE Internet Things J.*, vol. 3, no. 5, pp. 637–646, 2016.

[4] M. Satyanarayanan, "The emergence of edge computing," *Computer*, vol. 50, no. 1, pp. 30–39, 2017.

[5] C. P. Subbe, M. Kruger, P. Rutherford, and L. Gemmel, "Validation of a modified early warning score in medical admissions," *QJM*, vol. 94, no. 10, pp. 521–526, 2001.

[6] M. M. Churpek, T. C. Yuen, C. Winslow, D. O. Meltzer, M. W. Kattan, and D. P. Edelson, "Multicenter Comparison of Machine Learning Methods and Conventional Regression for Predicting Clinical Deterioration on the Wards," *Critical Care Medicine*, vol. 44, no. 2, pp. 368-374, 2016, doi: 10.1097/CCM.0000000000001571.

[7] A. E. W. Johnson *et al.*, "MIMIC-III, a freely accessible critical care database," *Sci. Data*, vol. 3, p. 160035, 2016.

[8] C. Rudin, "Stop explaining black box machine learning models for high stakes decisions and use interpretable models instead," *Nat. Mach. Intell.*, vol. 1, no. 5, pp. 206–215, 2019.

[9] Royal College of Physicians, *National Early Warning Score (NEWS) 2: Standardising the Assessment of Acute-Illness Severity in the NHS*. London, U.K.: RCP, 2017.

[10] L. Breiman, "Random forests," *Mach. Learn.*, vol. 45, no. 1, pp. 5–32, 2001.

[11] SQLite Development Team, "SQLite documentation." [Online]. Available: https://www.sqlite.org/docs.html

[12] Streamlit, "Streamlit documentation." [Online]. Available: https://docs.streamlit.io

[13] S. Ramírez, "FastAPI documentation." [Online]. Available: https://fastapi.tiangolo.com

[14] M. Agarwal, "Patient," Kaggle dataset (mitishaagarwal/patient). **Original source: M. Lee et al., 'WiDS (Women in Data Science) Datathon 2020: ICU Mortality Prediction,' PhysioNet, version 1.0.0, 2020, doi: 10.13026/vc0e-th79. License: PhysioNet Restricted Health Data License 1.5.0. Accessed: Oct. 9, 2026.**

[15] World Health Organization, *Ethics and Governance of Artificial Intelligence for Health*. Geneva, Switzerland: WHO, 2021.

[16] J. C. Mandel, D. A. Kreda, K. D. Mandl, I. S. Kohane, and R. B. Ramoni, "SMART on FHIR: A standards-based, interoperable apps platform for electronic health records," *J. Amer. Med. Inform. Assoc.*, vol. 23, no. 5, pp. 899–908, 2016.

[17] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proc. 20th Int. Conf. Artif. Intell. Statist. (AISTATS)*, 2017, pp. 1273–1282.